



Departamento de Informática e Matemática Aplicada – DIMAp

Spring AI – Parte 1: Conceitos, Prompt e Avaliação

Prof. Nélio Cacho

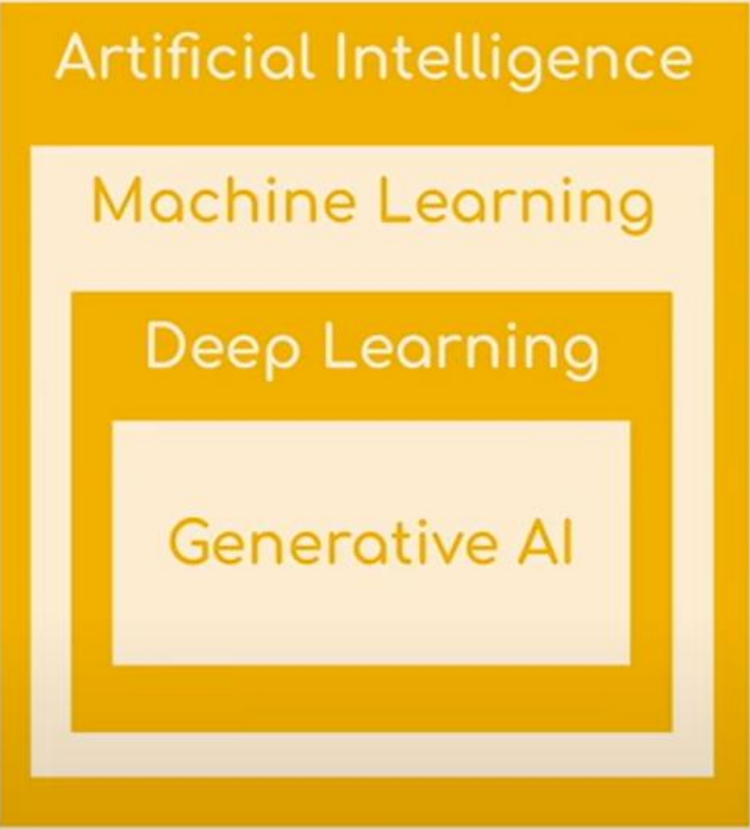
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Evolução da IA



Artificial Intelligence

Machine Learning

Deep Learning

Generative AI

1956 - Artificial Intelligence

CS field in developing intelligent machines that can think and learn like humans.

1997 - Machine Learning

Systems that learn from existing data to make decisions and predictions.

2012 - Deep Learning

Artificial Neural Networks, mimic the human brain structure, that can recognize complex patterns in data.

2021 - Generative Learning

Systems that can generate new data, such as images, videos, and audio.

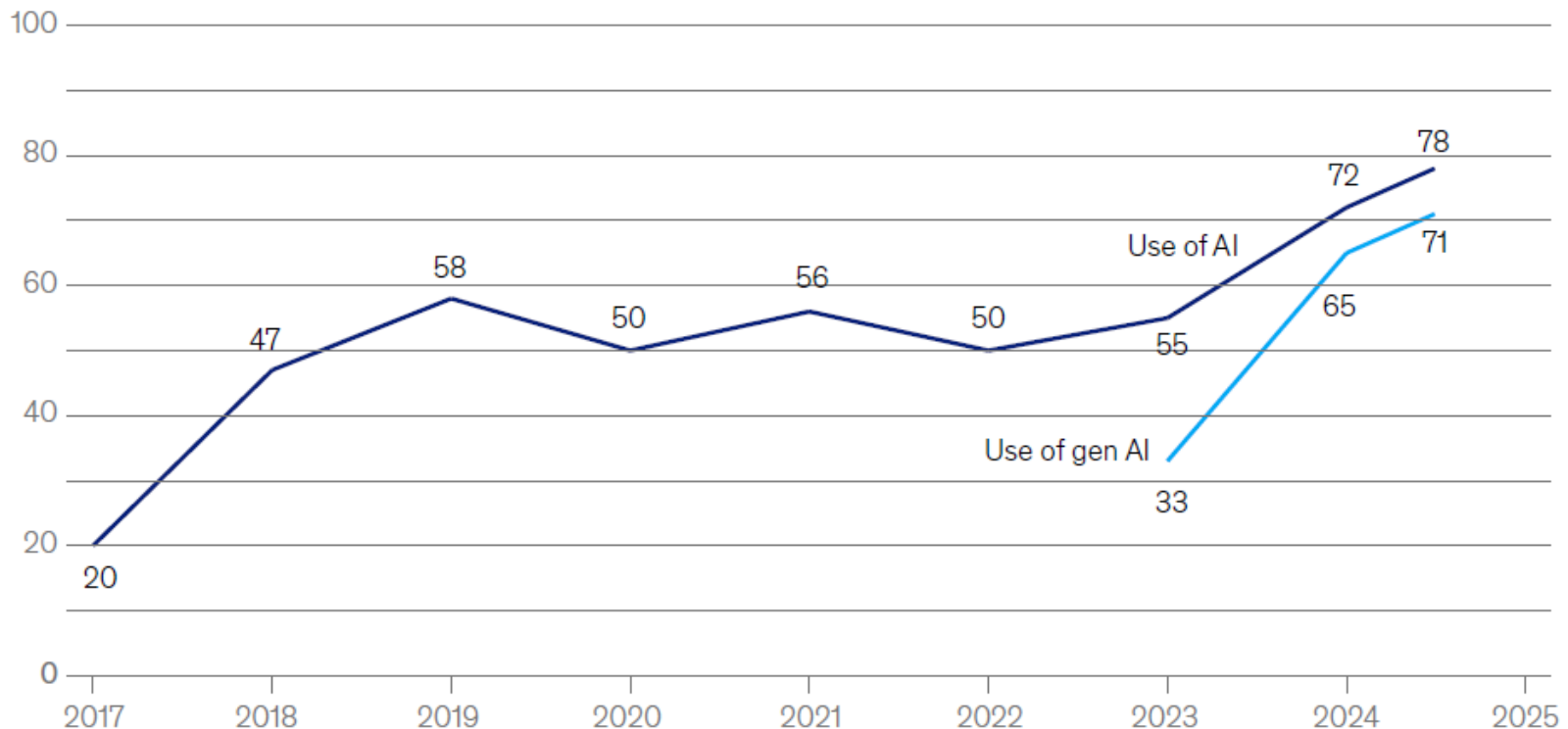
Estamos muito perto disso....



Motivação

Organizations' use of AI has accelerated markedly in the past year, after years of little meaningful change.

Organizations that use AI in at least 1 business function,¹ % of respondents



¹In 2017, the definition for AI use was using AI in a core part of the organization's business or at scale. In 2018–19, the definition was embedding at least 1 AI capability in business processes or products. Since 2020, the definition has been that the organization has adopted AI in at least 1 function.

Motivação

Most common ways companies are using AI

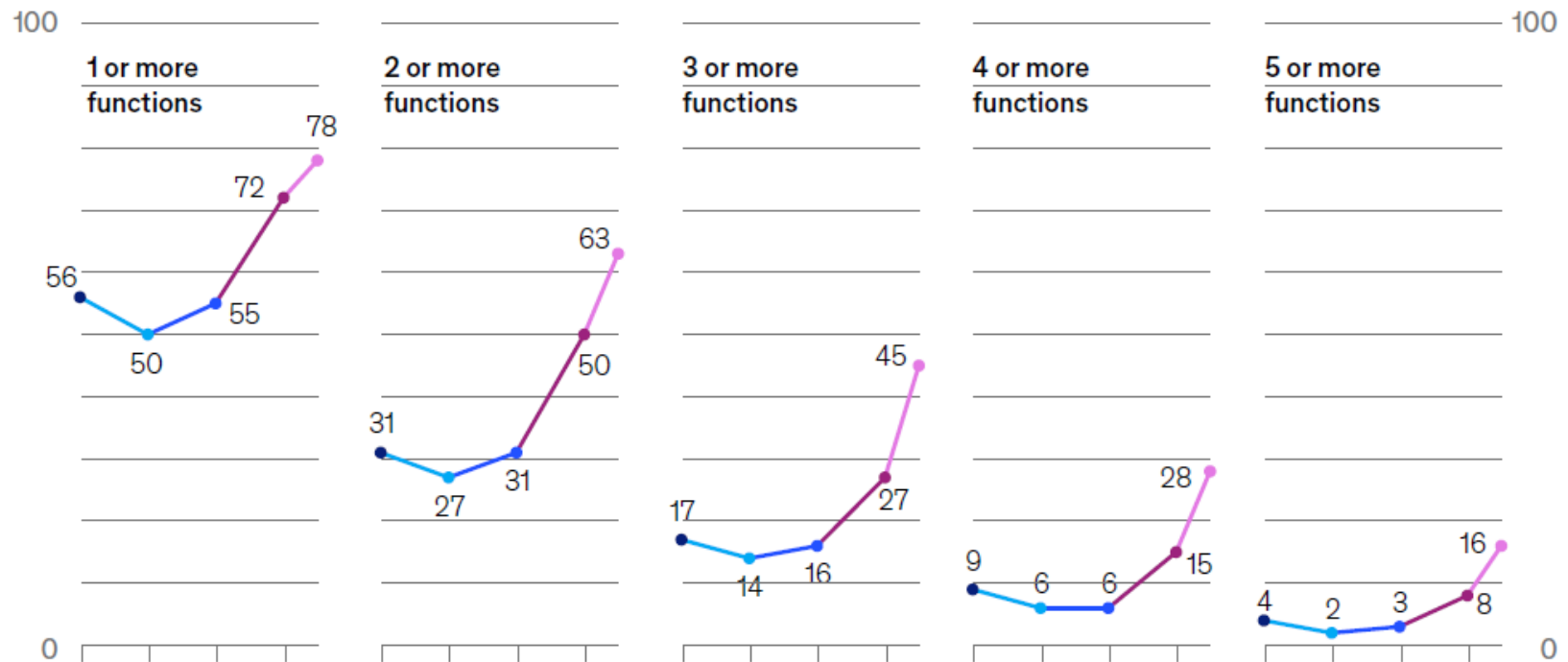


Motivação

Organizations are increasingly using AI in multiple functions.

Business functions at respondents' organizations that are using AI,¹ % of respondents

• 2021 2022 2023 First half of 2024 Second half of 2024



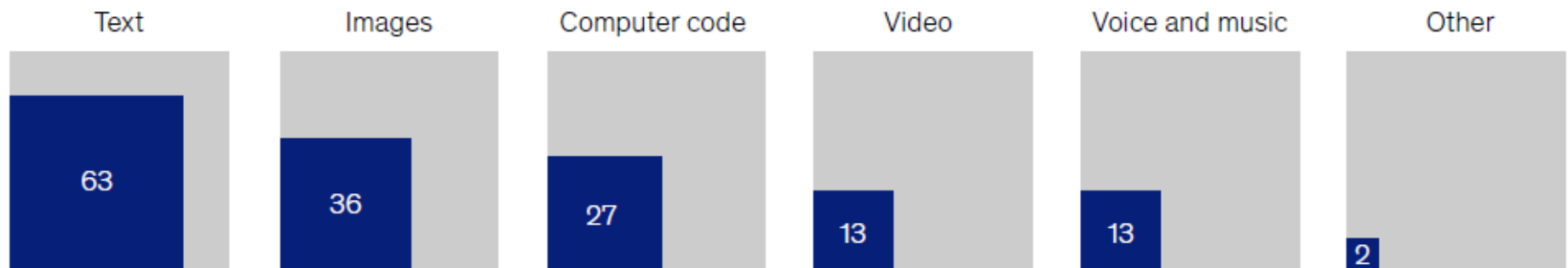
¹In 2021, n = 1,843; in 2022, n = 1,492; in 2023, n = 1,684; in Feb–Mar 2024, n = 1,363; in July 2024, n = 1,491. The survey question asks about 11 functions: HR; IT; manufacturing; marketing and sales; product and/or service development; risk, legal, and compliance; service operations; software engineering; strategy and corporate finance; supply chain/inventory management; and other corporate functions (eg, knowledge management).

Source: McKinsey Global Surveys on the state of AI, 2021–24

Motivação

While text is the type of content that organizations are most commonly creating with gen AI, they are also experimenting with other modalities.

Types of content generated by gen AI at respondents' organizations,¹ % of respondents



¹Only asked of respondents whose organizations regularly use gen AI in at least one function. Figures were calculated after removing the respondents who said "don't know."

Source: McKinsey Global Survey on the state of AI, 1,491 participants at all levels of the organization, July 16–31, 2024

Motivação

Global AI Adoption By Region



Contexto Geral de Desenvolvimento de Software com funcionalidades de IA



Cientista de Dados

Preparação dos Dados



Engenheiros de Aprendizado de Máquina

Treinamento do Modelo

Inferência ao Modelo



Engenheiros de Software e Desenvolvedores

API de Acesso

Message ChatGPT



Engenheiros de MLOps

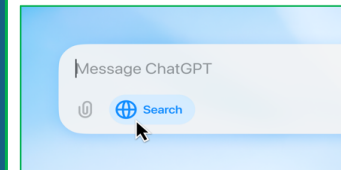
Infraestrutura Computacional

Contexto Geral de Desenvolvimento de Software com funcionalidades de IA

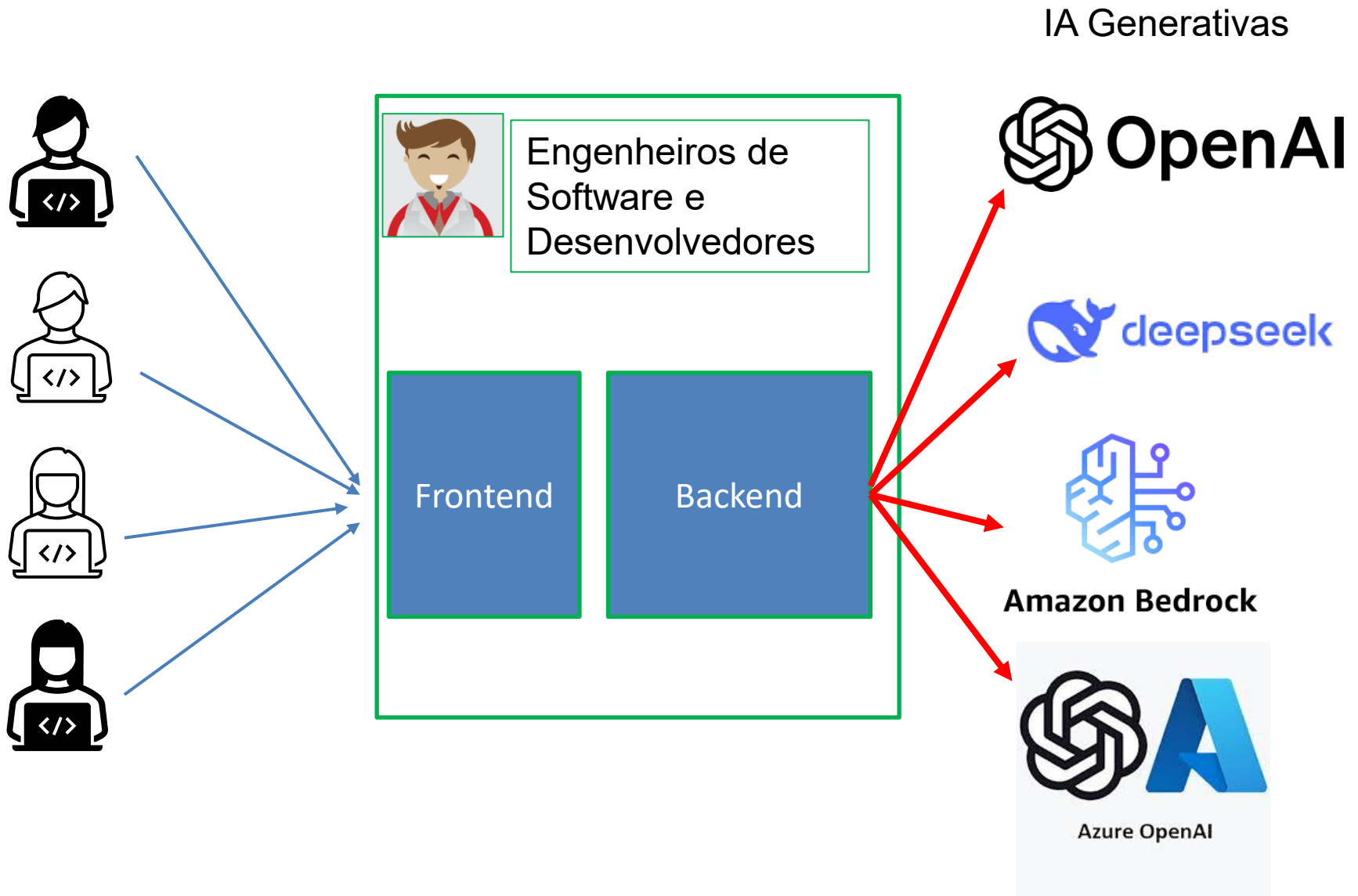


Engenheiros de
Software

Backend

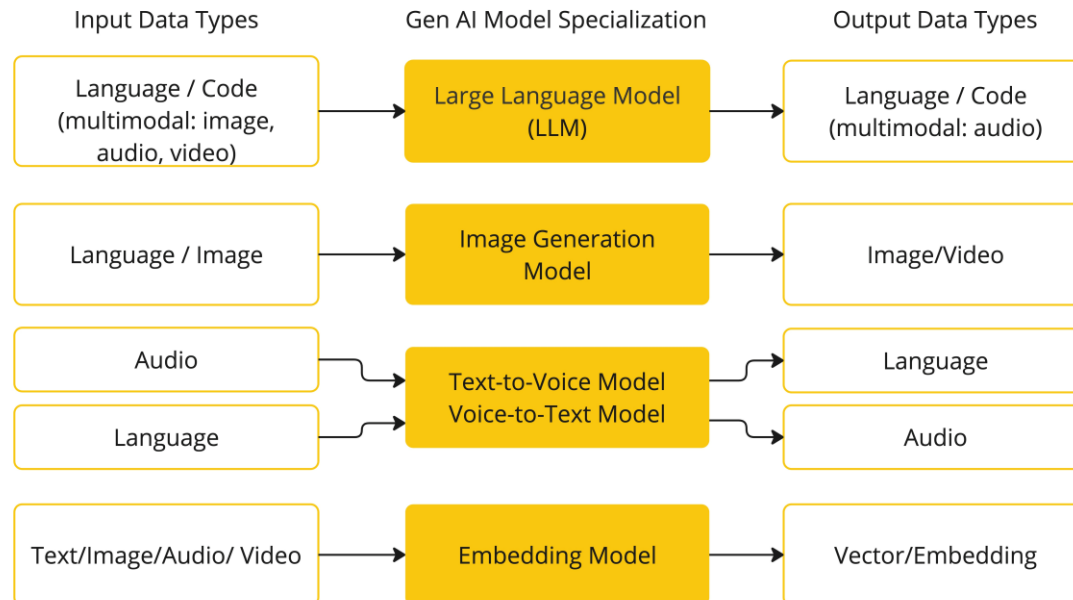


Contexto Geral de Desenvolvimento de Software com funcionalidades de IA



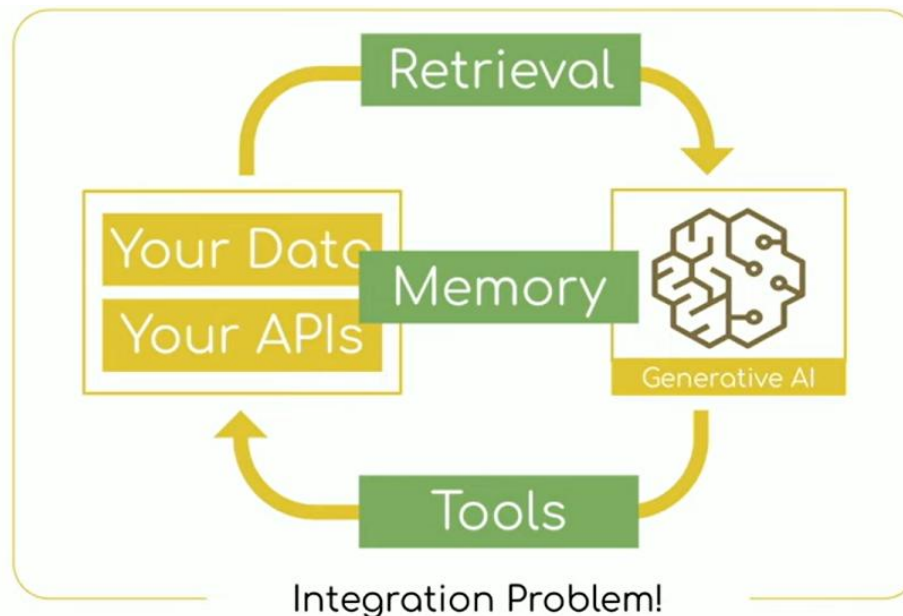
O que é uma IA Generativa?

- IA Generativa (GenAI) é uma classe de modelos de inteligência artificial capaz de criar novos conteúdos a partir de dados existentes. Esses conteúdos podem ser variados.
- Esses modelos são treinados com grandes volumes de dados e aprendem padrões para gerar saídas realistas e coerentes, mesmo quando recebem entradas inéditas.



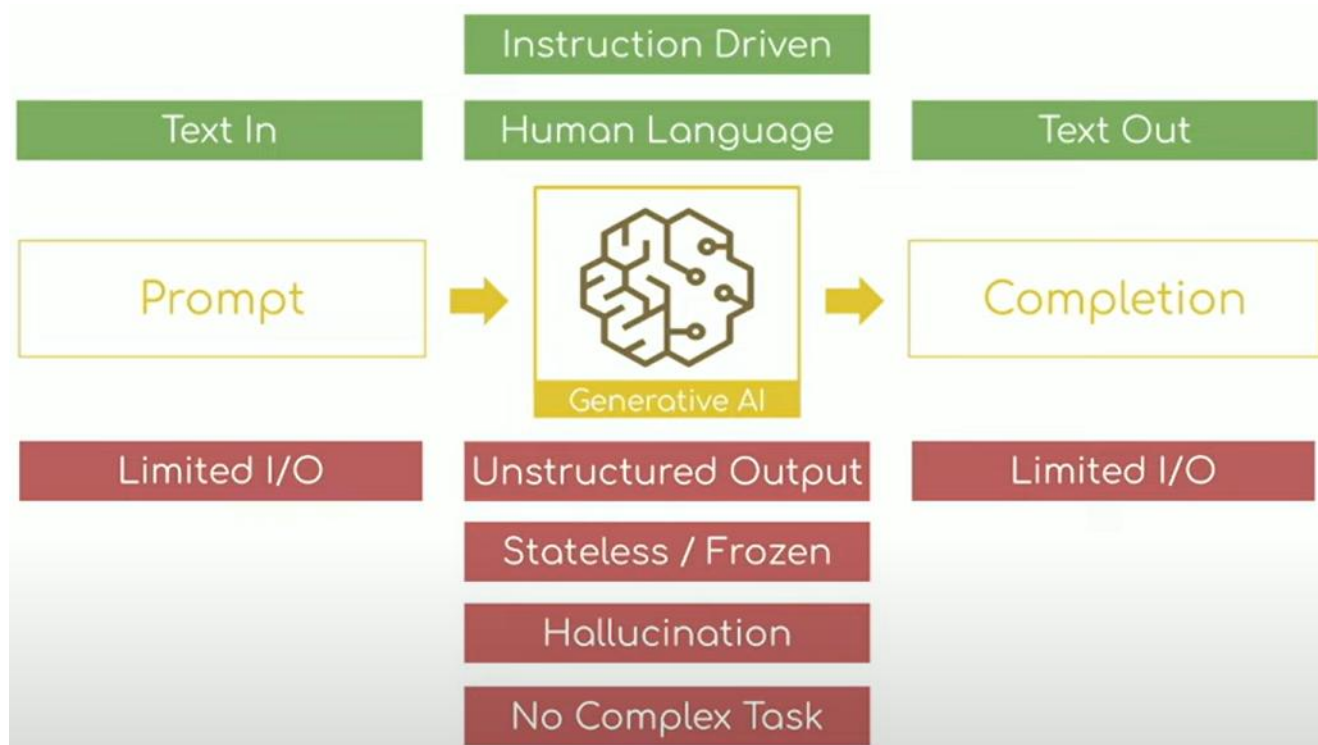
Desafio de Integração com IA Generativa

- A ideia geral de uma solução que usa IA Generativa, é que com base nos seus Dados e API, sua aplicação solicita a uma IA Generativa, que é Stateless, requisições. A IA Generativa produz novo conteúdo, no entanto, sua aplicação precisa gerenciar o estado caso seja preciso. **Isso gera um problema de integração.**



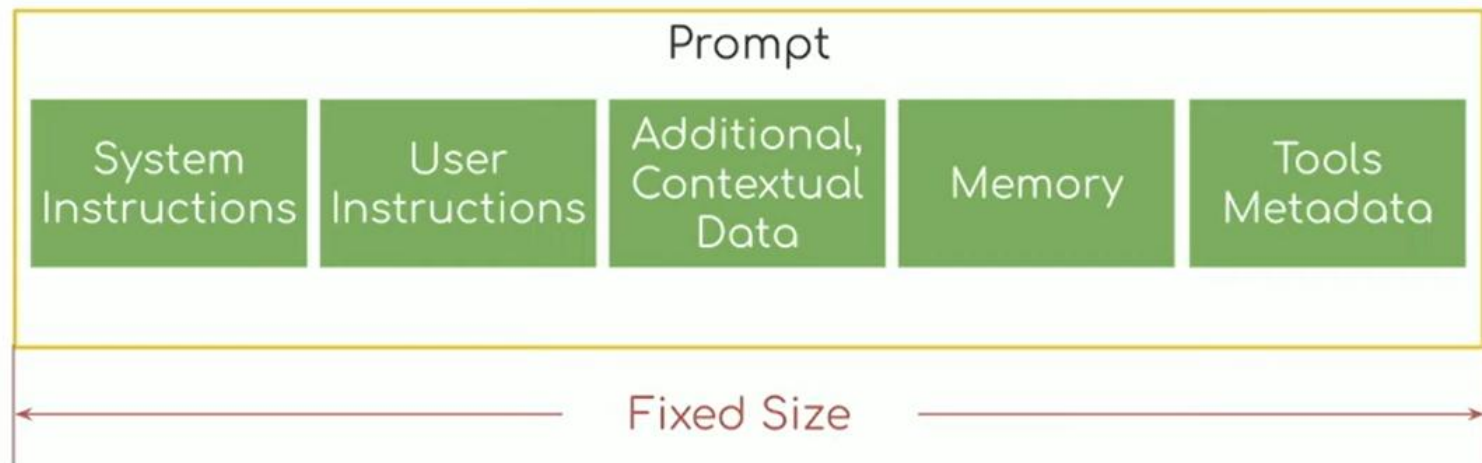
Desafio de Integração com IA Generativa

- Geralmente seu programa passa “Texto de Entrada” e “Recebe Texto” como retorno. Tudo isso tem restrições que precisam ser observadas:



Desafio de Integração com IA Generativa

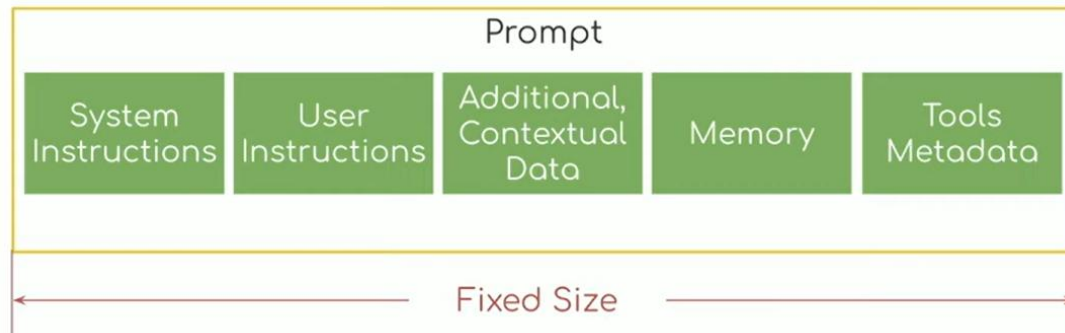
- Modelos de AI são bons o tanto quanto o **contexto** que é fornecido.
- Como modelos são *Stateless*, eles reagem ao Input fornecido. Neste sentido, o input deve conter várias informações adicionais para permitir um bom entendimento do modelo sobre sua requisição



Estrutura de um prompt

Um **prompt** em modelos de linguagem como os da OpenAI é dividido em cinco blocos principais que compõem o conteúdo total enviado ao modelo, todos dentro de um **tamanho fixo** de contexto (token limit):

1. **System Instructions** – Instruções de sistema: orientações que definem o comportamento do modelo (ex: “você é um assistente útil”).
 2. **User Instructions** – Instruções do usuário: a entrada direta do usuário, como perguntas ou comandos.
 3. **Additional, Contextual Data** – Dados contextuais adicionais: informações complementares relevantes para a tarefa (ex: histórico de mensagens ou documentos).
 4. **Memory** – Memória: conteúdo persistente aprendido sobre o usuário (caso esteja ativado).
 5. **Tools Metadata** – Metadados de ferramentas: informações sobre ferramentas externas que o modelo pode usar (como buscas ou cálculos).
- Todos esses elementos juntos compõem o prompt completo enviado ao modelo, que deve respeitar um limite de tamanho fixo (em tokens).

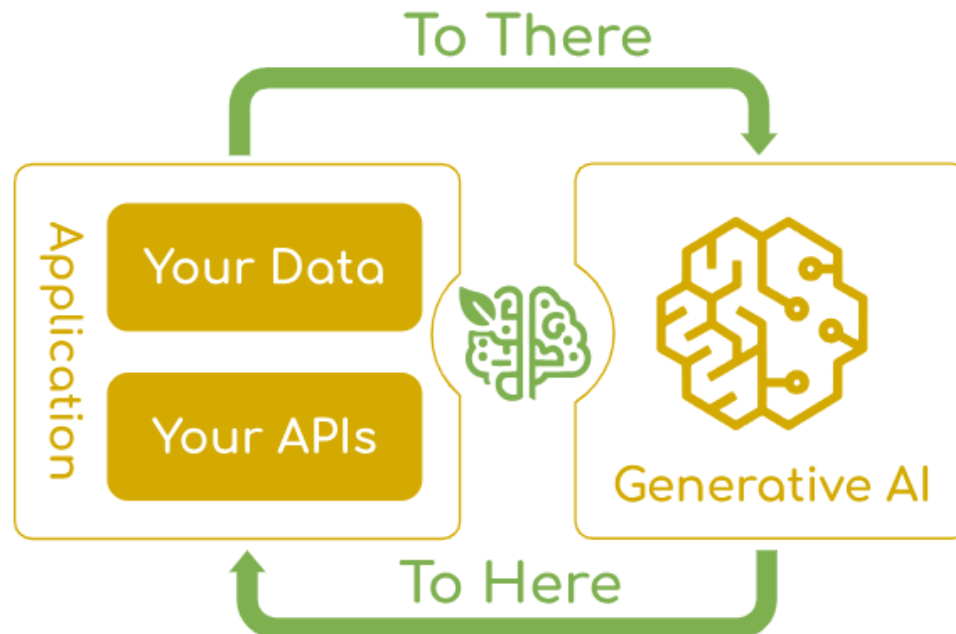


Por que IA Generativa é Única ?

- Vantagens:
 - Gera novos dados
 - Não requer treinamento
 - Usa linguagem Humana como Interface
 - Não precisa ser um Cientista de Dados ou Engenheiro de Aprendizado de Máquina para usar
- Desvantagens:
 - **Stateless API**
 - Não é treinada para usar os seus dados
 - Não conhece sua API
 - Possui limites de I/O para soluções pagas
 - Tendencioso a Alucinações

O que é Spring AI

Spring AI é um projeto do **Spring.IO** que **facilita a integração de inteligência artificial** — especialmente **modelos de linguagem (LLMs)** — em **aplicações Java** de forma padrão, segura e produtiva.



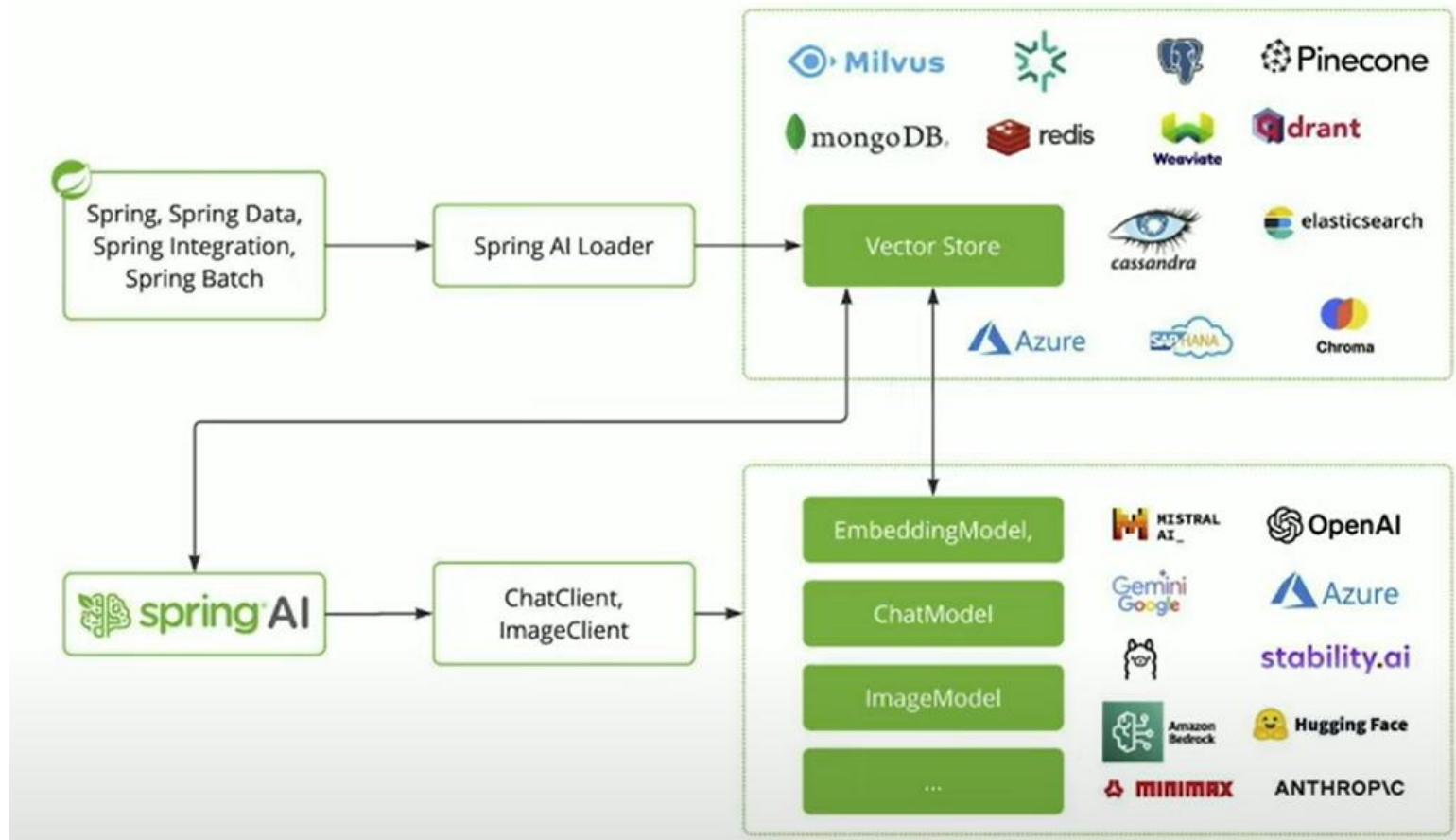
O que é Spring AI

O Spring AI cria uma camada de abstração para que você possa:

- Acessar diferentes provedores de IA (OpenAI, Hugging Face, Azure OpenAI, etc.);
- Trabalhar com **modelos de texto, geração de imagens, embeddings, vetores**;
- Construir **fluxos inteligentes** de interação, como **chatbots, agentes autônomos, RAG**;
- Fazer isso tudo com o estilo conhecido do Spring: **anotações, beans, configuração declarativa**, etc.

Suporte Fornecido pelo Spring AI

- Spring AI fornece suporte para integração com uma ampla gama de modelos.



O que é Spring AI NÃO É!!!

Spring AI não foi desenvolvido para:

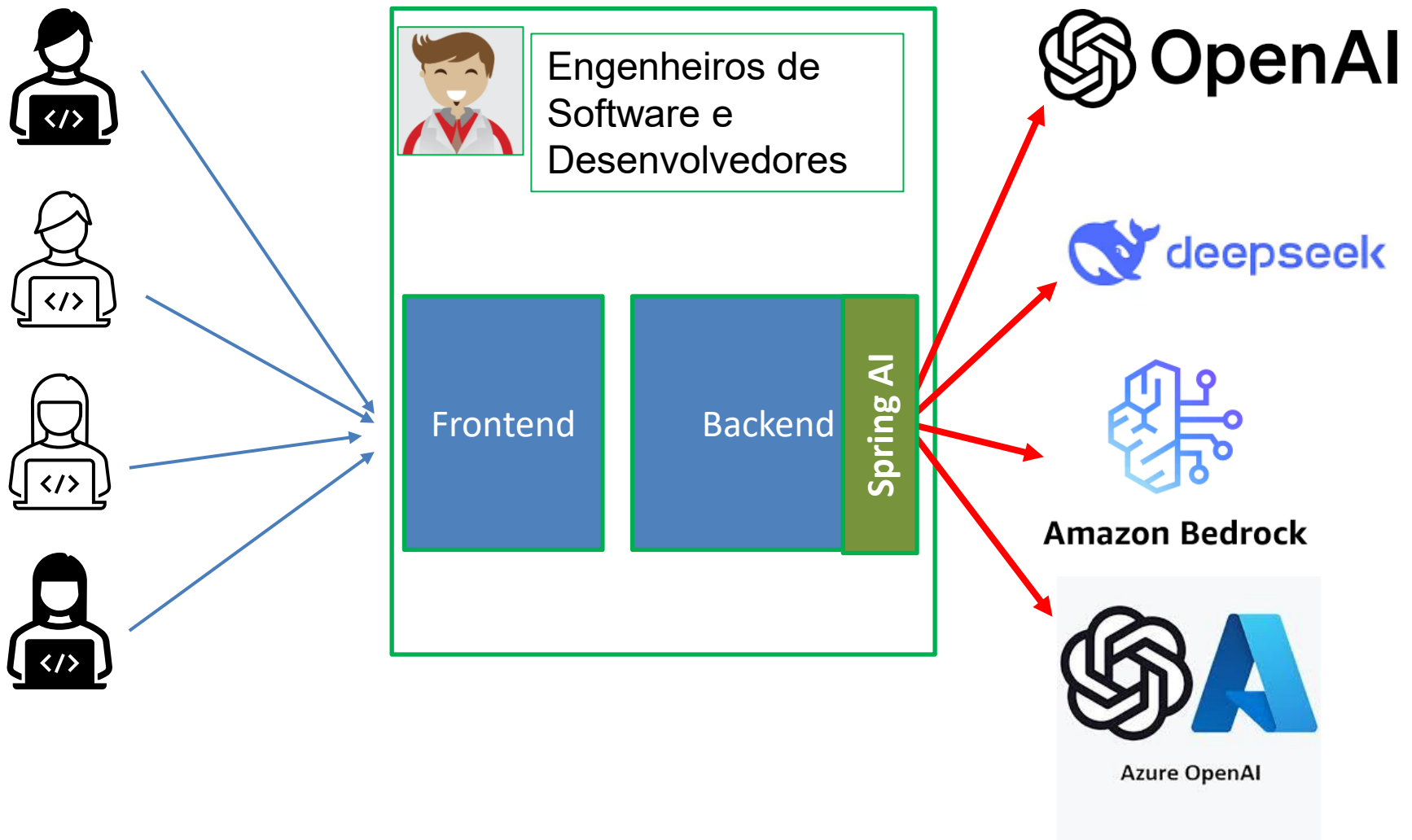
— Gerar seu próprio *Foundational Models*.

- **Foundational Models** (ou **Modelos Fundamentais**) são **modelos de inteligência artificial extremamente grandes e genéricos**, treinados em **enormes quantidades de dados** (textos, imagens, códigos, etc.), que podem ser **adaptados para uma variedade enorme de tarefas**.

— Realizar fine-tuning the modelos existentes.

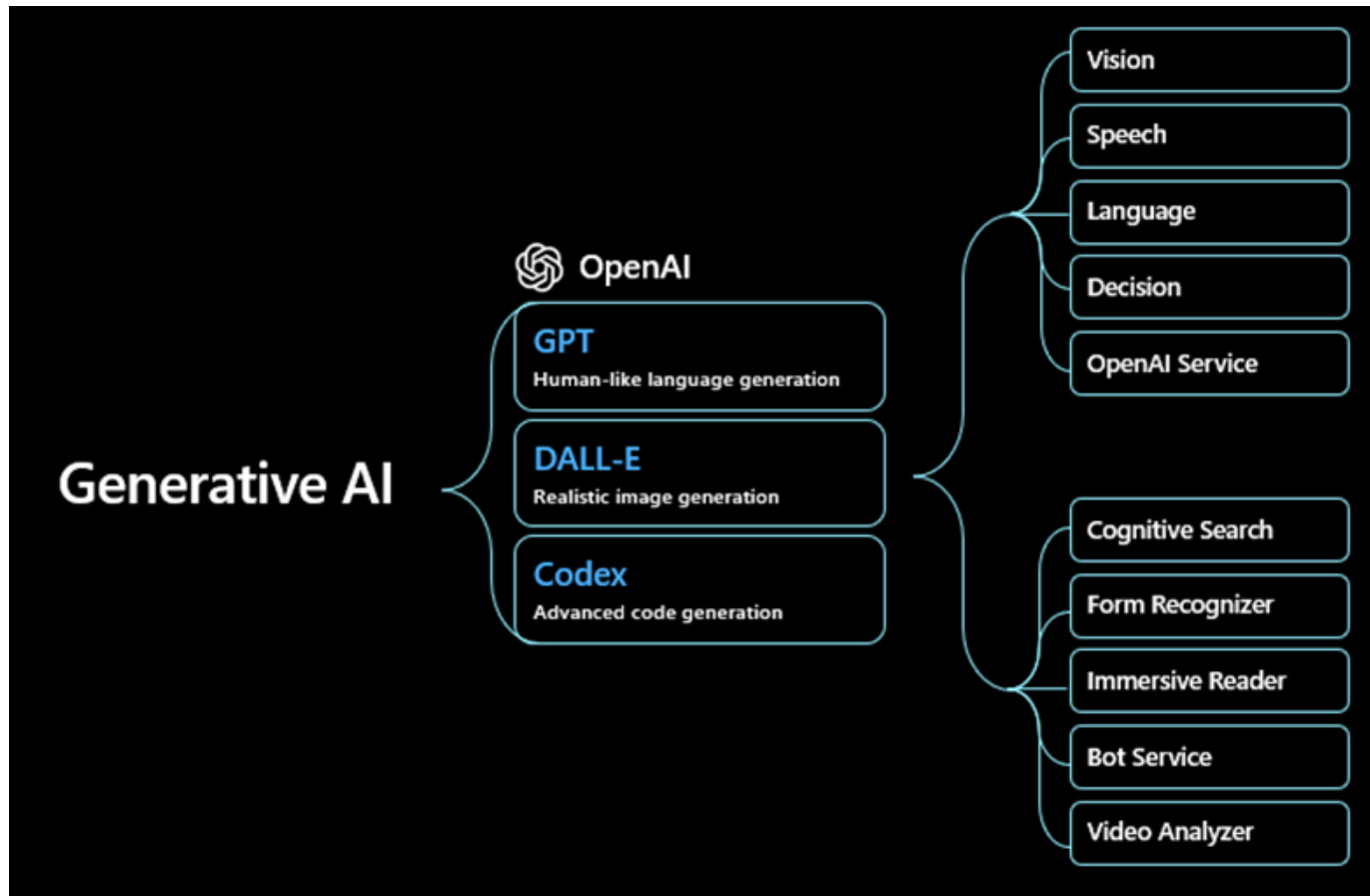
- **Fine-tuning** é o processo de **pegar um modelo já treinado e reajustá-lo com um novo conjunto de dados** específico para **especializar** ele em uma tarefa, domínio ou linguagem.

Onde Entra o Spring AI na nossa Arquitetura?



Qual modelo usar?

- A primeira coisa a fazer é escolher o modelo:



Qual modelo usar?

- Além da OpenAI, tem vários outros fornecedores:
 - **Amazon Bedrock** – Um serviço de IA oferecido pela plataforma de nuvem da Amazon, com modelos como Claude, Llama 3.1, Mistral e Titan.
 - **Anthropic AI** – Um serviço de IA fundado por ex-membros da OpenAI, oferecendo a família de modelos Claude.
 - **Azure OpenAI** – Essencialmente o mesmo conjunto de modelos da OpenAI, mas oferecido através da plataforma de computação da Microsoft, o Azure.
 - **Gemini** – Um serviço de IA oferecido pela plataforma de nuvem do Google.
 - **HuggingFace** – Oferece um repositório com mais de 300.000 modelos disponíveis. A integração do Spring AI com o HuggingFace funciona com a API baseada na nuvem do HuggingFace.
 - **IBM WatsonX** – A oferta da IBM de LLMs (Modelos de Linguagem de Grande Escala) através de sua plataforma de nuvem WatsonX.
 - **MiniMax** – Um serviço de IA chinês que oferece vários modelos, incluindo modelos multilíngues.
 - **MistralAI** – Uma empresa de IA fundada por ex-funcionários da Meta e do Google que oferece diversos LLMs muito competentes, incluindo o popular modelo Mistral 7B.
 - **Ollama** – Uma opção para executar gratuitamente diversos modelos de código aberto localmente no seu próprio hardware, incluindo vários modelos populares também oferecidos por serviços em nuvem.

Critérios de Escolha

Com tantas opções, como escolher qual serviço e modelo usar? Embora não exista uma forma definitiva de escolher um modelo, há alguns critérios importantes a considerar ao selecionar um provedor e um modelo:

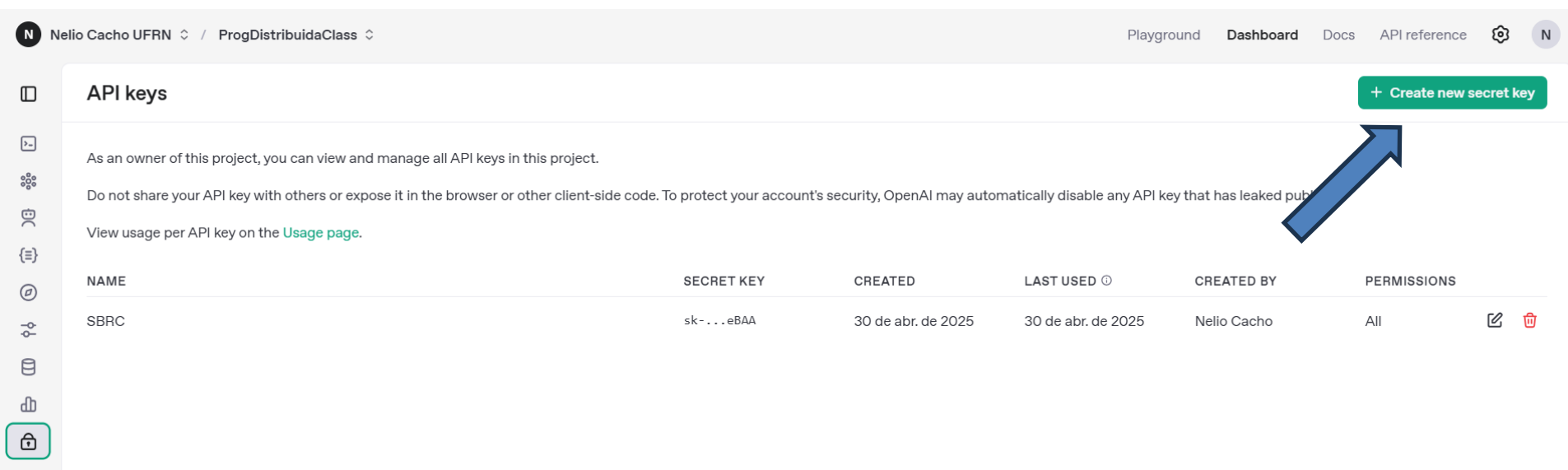
- **Preço** – A maioria das opções, especialmente as baseadas em nuvem, requerem pagamento. Embora muitas delas tenham preços bem acessíveis, você deve considerar como sua escolha afetará o orçamento. No entanto, também existem opções gratuitas, como os modelos disponibilizados pelo Ollama e Gemini*.
- **Janela de contexto** – Ao processar um prompt e fornecer uma resposta, tanto o prompt quanto a resposta são divididos em unidades menores chamadas *tokens*. Como os tokens são criados não é tão importante quanto a quantidade permitida no prompt e na resposta. Diferentes modelos possuem diferentes janelas de contexto, variando de alguns milhares a milhões de tokens por interação. Esses limites não são um grande problema ao fazer perguntas simples, mas à medida que você adiciona histórico de conversas e contexto de documentos aos seus prompts, é importante garantir que não ultrapassem os limites do modelo.

Critérios de Escolha

- **Treinamento** – A principal diferença entre os modelos está nos dados com os quais foram treinados. Alguns foram treinados com conjuntos de dados muito maiores que outros, enquanto outros foram treinados com dados menores, porém mais especializados. Além disso, a data até a qual um modelo foi treinado influencia sua capacidade de fornecer respostas com base em informações mais atuais.
- **Capacidades** – Alguns provedores e modelos de IA oferecem recursos adicionais, como a capacidade de transmitir respostas em tempo real ou integrar conteúdo de funções de aplicativos. Como esses recursos não são padrão em todos os LLMs e provedores, é importante considerar se você precisará dessas funcionalidades extras.

Usando Spring AI com OpenAI Models

- Primeiro é preciso obter uma API Key
<https://platform.openai.com/>

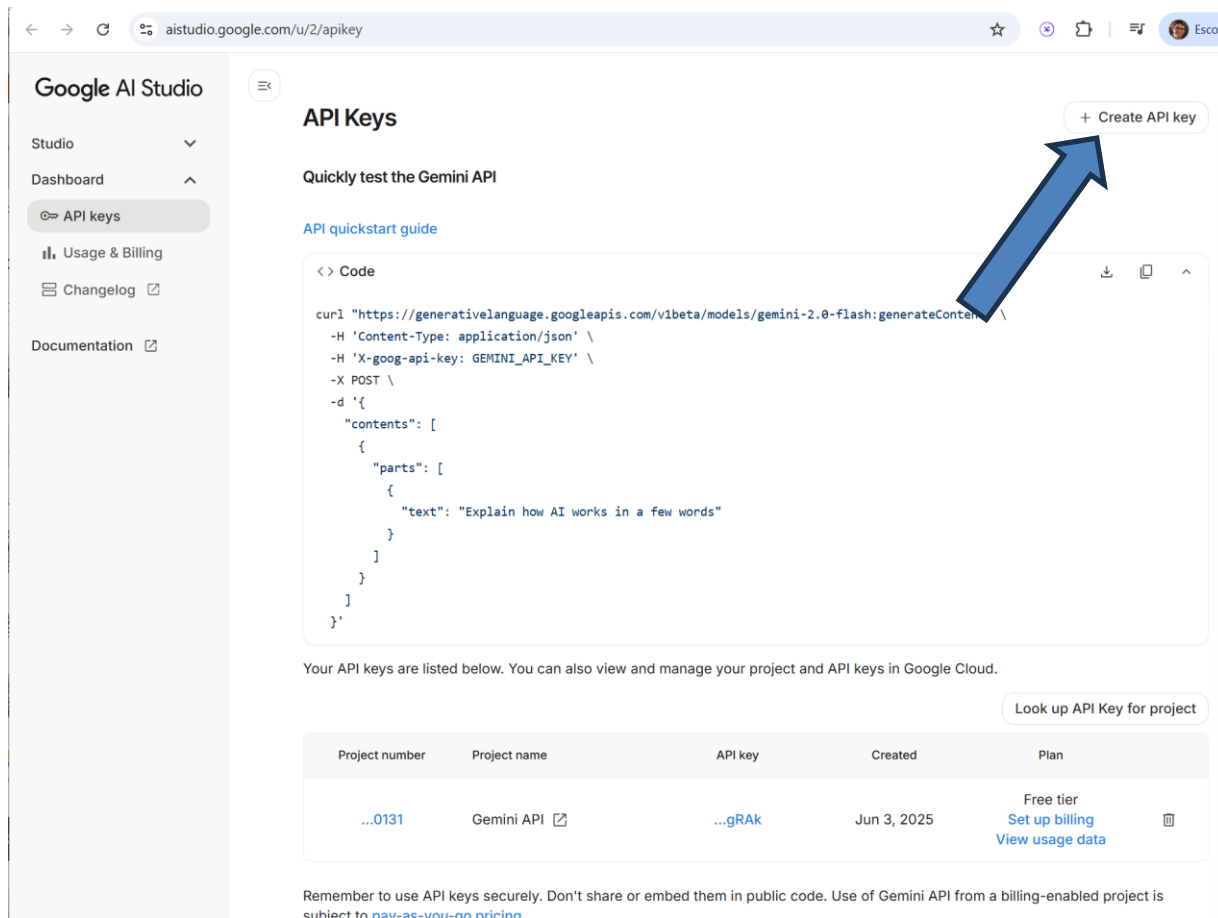


The screenshot displays the OpenAI API keys management interface. At the top, there's a navigation bar with links to Playground, Dashboard, Docs, and API reference. The main heading is "API keys". Below this, there's a green button labeled "+ Create new secret key". A blue arrow points to this button. The page also contains a table of existing API keys.

NAME	SECRET KEY	CREATED	LAST USED	CREATED BY	PERMISSIONS
SBRC	sk-...eBAA	30 de abr. de 2025	30 de abr. de 2025	Nelio Cacho	All

Usando Spring AI com Gemini

- Primeiro é preciso obter uma chave em:
- <https://aistudio.google.com/u/2/apikey>



The screenshot shows the Google AI Studio interface. On the left is a sidebar with navigation links: Studio, Dashboard, API keys (selected), Usage & Billing, Changelog, and Documentation. The main content area is titled 'API Keys' and includes a 'Quickly test the Gemini API' section with a 'Code' tab containing a cURL command. A blue arrow points to the '+ Create API key' button in the top right. Below the code section, there is a table listing API keys. The table has columns for Project number, Project name, API key, Created, and Plan. One key is listed with project number ...0131, name Gemini API, key ...gRAk, created on Jun 3, 2025, and on the Free tier.

Project number	Project name	API key	Created	Plan
...0131	Gemini API	...gRAk	Jun 3, 2025	Free tier Set up billing View usage data

Criar um projeto Spring

start.spring.io



Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.5.0 (SNAPSHOT)

☐ 3.5.0 (RC1)

☐ 3.4.6 (SNAPSHOT)

☒ 3.4.5

☐ 3.3.12 (SNAPSHOT)

☐ 3.3.11

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☒ 24 ☐ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

OpenAI AI

Spring AI support for ChatGPT, the AI language model and DALL-E, the Image generation model from OpenAI.

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot Actuator OPS

Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

Criar um projeto Spring com as seguintes dependências

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
  <version>3.4.5</version>
</dependency>

<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
```

Defina as propriedades do seu projeto no `application.properties` para o OpenAI

- É preciso informar a API Key no seu `application.properties`. No entanto, não é uma boa prática colocar sua KEY de forma explícita. Portanto, deve-se usar uma variável de ambientes.

```
spring.application.name=prompt  
spring.ai.openai.api-key=${OPENAI_API_KEY}  
spring.ai.openai.model=gpt-3.5-turbo
```

Defina as propriedades do seu projeto no `application.Properties` para Gemini

- É preciso informar a API Key no seu `application.properties`. No entanto, não é uma boa prática colocar sua KEY de forma explícita. Portanto, deve-se usar uma variável de ambientes.

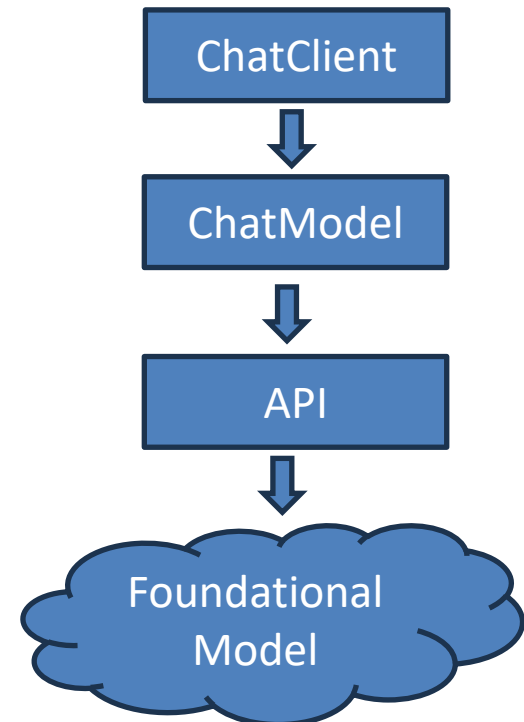
```
spring.ai.openai.api-key=${APIKEYGemini}  
spring.ai.openai.base-url=https://generativelanguage.googleapis.com/v1beta/openai  
spring.ai.openai.chat.completions-path=/chat/completions  
spring.ai.openai.chat.options.model=gemini-2.5-flash
```

Model Beans

- Model Beans representam os Beans que permitem a integração do Spring com os modelos de IA.
 - Diferentes tipos:
 - ChatModels
 - ImageModels
 - EmbeddingModels
 - etc;
 - Model Beans são auto-configurados via propriedades contidas nos arquivos de propriedades.

Client Beans

- Spring AI oferece um `ChatClient` interface para facilitar o uso do `ChatModel`.
 - Fornece uma interface mais fácil de usar
 - Fornece métodos síncronos ou assíncronos de invocação
 - Fornece Advisors
- Não é Auto Configurado
 - Use um `ChatClient.Builder` para criar um `ChatClient` de forma configurável



Invocando a API do OpenAI

- Vamos usar o ChatClient sem configuração

```
package br.imd.ufrn.prompt;  
  
public interface ChatService {  
    String getResposta(String pergunta);  
}
```

```
@Service  
public class OpenAIChatService implements ChatService {  
    private final ChatClient chatClient;  
  
    public OpenAIChatService(ChatClient.Builder chatClientBuilder) {  
        this.chatClient = chatClientBuilder.build();  
    }  
  
    @Override  
    public String getResposta(String pergunta) {  
        return chatClient.prompt().user(pergunta).call().content();  
    }  
}
```

Invocando a API do OpenAI

- Pode-se usar ChatOptions para definir opções de operação do ChatClient, como o modelo a ser utilizado.

```
public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {  
    ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-4o-mini").build();  
    this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();  
}
```


Vamos definir um Controller

```
@RestController
```

```
public class PromptController {
```

```
    private final ChatService chatService;
```

```
    public PromptController(ChatService chatService) {  
        this.chatService = chatService;  
    }
```

```
    http://127.0.0.1:8080/prompt
```

```
@GetMapping("prompt")
```

```
public String getMethodName(@RequestParam String pergunta) {  
    return chatService.getResposta(pergunta);  
}
```

```
}
```

Resultado...

Primeira Requisição:

← → ↺ `http://localhost:8080/prompt?pergunta=Meu Nome é Nelio, qual o meu nome`

Seu nome é Nélio. Como posso ajudar você hoje?

Segunda Requisição:

← → ↺ `http://localhost:8080/prompt?pergunta=Qual o meu nome`

Desculpe, mas não posso saber qual é o seu nome. Se você quiser, pode me dizer seu nome!

Que Tal usar um modelo OpenSource?

- Pode-se usar modelos de IA generativa localmente no seu computador de forma **gratuita e offline**, especialmente se usar modelos **open-source**. Existem algumas opções disponíveis:
 - Ollama: <https://ollama.com/>
 - LM Studio: <https://lmstudio.ai>
 - Com Docker....

Ollama

- O **Ollama** é uma ferramenta que permite **rodar modelos de linguagem (LLMs) localmente** no seu computador, sem depender da nuvem. É ideal para quem quer usar IA generativa com **privacidade, offline e sem custos de API**.
- **Como instalar?**
 - Acesse: <https://ollama.com>
 - Escolha sua plataforma (Windows, macOS, Linux)
 - Instale e abra o terminal.

Ollama

- Baixando um modelo: `ollama pull <model>`

```
nelio@LaptopNelioSM:~$ ollama pull gemma3:12b
pulling manifest
pulling e8ad13eff07a: 100%
pulling e0a42594d802: 100%
pulling dd084c7d92a3: 100%
pulling 3116c5225075: 100%
pulling 6819964c2bcf: 100%
verifying sha256 digest
writing manifest
success
```

	8.1 GB
	358 B
	8.4 KB
	77 B
	490 B

- Listando modelos Existentes:

```
nelio@LaptopNelioSM:~$ ollama list
```

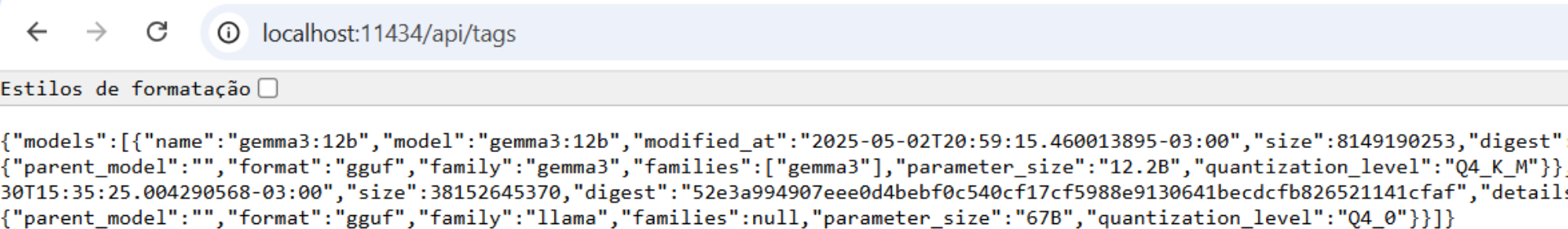
NAME	ID	SIZE	MODIFIED
gemma3:12b	f4031aab637d	8.1 GB	46 seconds ago
deepseek-llm:67b	52e3a994907e	38 GB	2 days ago

Ollama

- Rodando um Modelo

```
nelio@LaptopNelioSM:~$ ollama run deepseek-llm:67b
Error: model requires more system memory (33.5 GiB) than is available (18.8 GiB)
nelio@LaptopNelioSM:~$ ollama run gemma3:12b
>>> Eu sou Nélío, quem sou eu?
Olá, Nélío!
```

- Expondo uma porta Localhost:11434



← → ↻ ⓘ localhost:11434/api/tags

Estilos de formatação ☐

```
{
  "models": [
    {
      "name": "gemma3:12b",
      "model": "gemma3:12b",
      "modified_at": "2025-05-02T20:59:15.460013895-03:00",
      "size": 8149190253,
      "digest": "30T15:35:25.004290568-03:00",
      "parent_model": "",
      "format": "gguf",
      "family": "gemma3",
      "families": ["gemma3"],
      "parameter_size": "12.2B",
      "quantization_level": "Q4_K_M"
    },
    {
      "name": "llama3.1:67b",
      "model": "llama3.1:67b",
      "modified_at": "2025-05-02T20:59:15.460013895-03:00",
      "size": 38152645370,
      "digest": "52e3a994907eee0d4bebf0c540cf17cf5988e9130641becdcfb826521141cfaf",
      "parent_model": "",
      "format": "gguf",
      "family": "llama",
      "families": null,
      "parameter_size": "67B",
      "quantization_level": "Q4_0"
    }
  ]
}
```

- Parando um modelo:

```
nelio@LaptopNelioSM:~$ ollama stop gemma3:12b
```

Como acessar modelos do Ollama via Spring AI?

- É preciso usar uma dependência específica:

```
<dependency>  
  <groupId>org.springframework.ai</groupId>  
  <artifactId>spring-ai-ollama-spring-boot-starter</artifactId>  
  <version>1.0.0-M6</version>  
</dependency>
```

- Configurar o servidor Ollama:

```
spring.ai.ollama.base-url=http://localhost:11434  
spring.ai.ollama.chat.model=mistral:latest
```

Como acessar modelos do Ollama via Spring AI?

- O código é o mesmo do Serviço anterior para OpenAI:

```
@Service
@Primary
public class OllamaChatService implements ChatService {
    private final ChatClient chatClient;

    public OllamaChatService(ChatClient.Builder chatClientBuilderOllamaBuilder) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gemma3:12b").build();
        this.chatClient = chatClientBuilderOllamaBuilder.defaultOptions(chatOptions).build();
    }

    @Override
    public String getResposta(String pergunta) {
        return chatClient.prompt().user(pergunta).call().content();
    }
}
```


Como acessar modelos do Ollama via Spring AI?

- Resultado:

← → ↻ <http://localhost:8080/prompt?pergunta=Meu nome é Nelio, qual é meu nome?>

Seu nome é Nelio. Você mesmo me disse! 😊

← → ↻ <http://localhost:8080/prompt?pergunta=qual é meu nome?>

Como sou um modelo de linguagem, não tenho como saber seu nome. Você não me disse! 😊 Se quiser, pode me dizer seu nome!

Como avaliar a resposta da sua IA generativa?

- O uso de IAs generativas é uma mudança de paradigma no desenvolvimento de soluções de software.
- No entanto, isso vem acompanhado de um risco: **Alucinações.**
- Alucinação de uma IA generativa é quando o modelo "inventa" ou gera informações incorretas, inconsistentes ou sem base na realidade, mesmo que essas informações pareçam plausíveis ou confiáveis.

Exemplos de alucinação

- **Citar fontes inexistentes:** “Segundo o livro X de 2022...” — mas o livro nunca foi publicado.
- **Responder com dados falsos:** Dizer que a capital da Austrália é Sydney (quando é Canberra).
- **Criar códigos ou fórmulas erradas,** mesmo que pareçam corretas à primeira vista.
- **Misturar fatos reais com invenções,** dificultando a distinção entre o verdadeiro e o falso.

Por que Alucinações Acontecem?

- A IA, como o ChatGPT, não entende o mundo da forma como os humanos entendem. Ela prediz palavras com base em padrões linguísticos, não com base em conhecimento validado.
- Se falta **contexto** ou se a pergunta é ambígua, a IA pode "preencher lacunas" com algo que pareça adequado, mesmo que não seja verdadeiro.

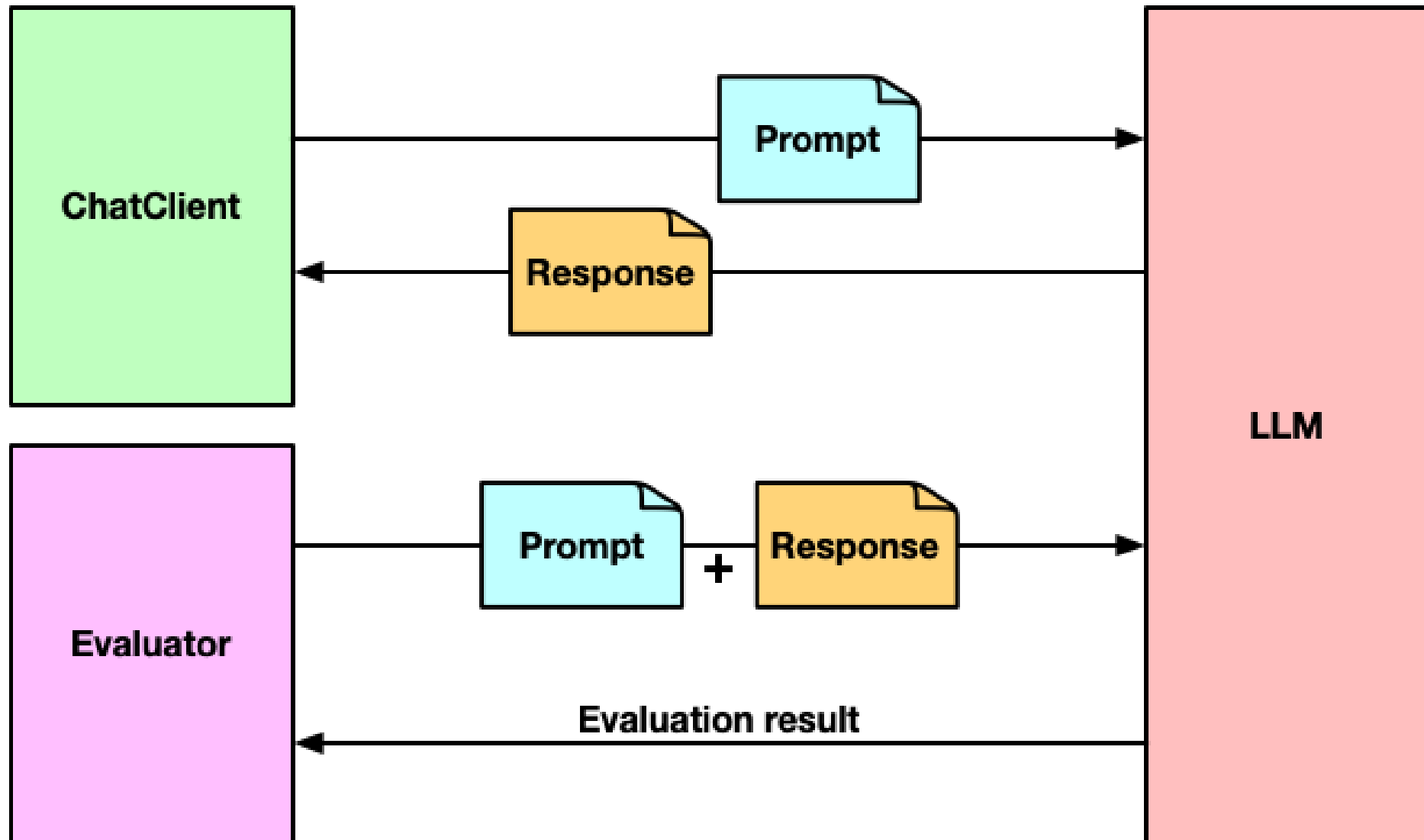
Natureza não-determinística

- Lembrando que uma IA generativa fornece resposta de forma não-determinísticas.
- Essa natureza não-determinística compromete o desenvolvimento de teste de software que trabalham com a abordagem “assert equals”.

Como Avaliar Resposta no Spring AI?

- Felizmente, o Spring AI oferece outra forma de decidir se a resposta gerada é aceitável: os **Avaliadores (Evaluators)**.
- Um avaliador recebe o texto do usuário a partir do prompt enviado ao LLM, junto com o conteúdo da resposta, e decide se o conteúdo da resposta atende a certos critérios.
- Por trás dos panos, um avaliador pode ser implementado de qualquer maneira que seja adequada ao tipo de avaliação que ele realiza.

Como Avaliar Resposta no Spring AI?



Como Avaliar Resposta no Spring AI?

- Atualmente o Spring AI suporta dois tipos de Evaluators:
 - **RelevancyEvaluator:** Esse avaliador verifica se a resposta gerada é relevante em relação à pergunta original e ao contexto fornecido. Ele utiliza o próprio modelo de linguagem para avaliar se a resposta está alinhada com a consulta e o contexto. A avaliação retorna um resultado booleano (true ou false) indicando se a resposta é considerada relevante.
 - **FactCheckingEvaluator:** Esse avaliador verifica a precisão factual da resposta gerada, comparando-a com um documento de contexto fornecido. Ele é útil para detectar e reduzir alucinações, garantindo que as afirmações feitas pela IA sejam logicamente suportadas pelo contexto. Modelos menores e mais eficientes, como o Bespoke Minicheck, podem ser utilizados para essa verificação, ajudando a reduzir os custos em comparação com modelos maiores.

Usando RelevancyEvaluator

```
@Service
@Primary
public class OpenAIChatService implements ChatService {
    private final ChatClient chatClient;
    private RelevancyEvaluator relevancyEvaluator;

    public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-3.5-turbo").build();
        this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();
        this.relevancyEvaluator = new RelevancyEvaluator(chatClientBuilderOpenAI);
    }

    @Override
    public String getResposta(String pergunta) {
        String resposta = chatClient.prompt().user(pergunta).call().content();

        EvaluationRequest evaluationRequest = new EvaluationRequest(pergunta, resposta);
        EvaluationResponse response = relevancyEvaluator.evaluate(evaluationRequest);
        return resposta + "\n" +
            "Score: " + response.getScore() + "\n" +
            "Passou: " + response.isPass() + "\n";
    }
}
```

Usando RelevancyEvaluator

← → ↻  localhost:8080/prompt?pergunta=qual a capital do RN, Brazil

A capital do Rio Grande do Norte, no Brasil, é a cidade de Natal. Score: 1.0 Passou: true

Usando FactCheckingEvaluator:

- O FactCheckingEvaluator do Spring AI funciona de maneira semelhante ao RelevancyEvaluator, exceto que, em vez de pedir ao LLM para julgar a relevância da resposta em relação à pergunta, ele solicita que o LLM julgue se a resposta responde corretamente à pergunta.

Usando FactCheckingEvaluator:

```
public class OpenAIChatService implements ChatService {
    private final ChatClient chatClient;
    private RelevancyEvaluator relevancyEvaluator;
    private FactCheckingEvaluator factCheckingEvaluator;

    public OpenAIChatService(ChatClient.Builder chatClientBuilderOpenAI) {
        ChatOptions chatOptions = ChatOptions.builder().model(model:"gpt-3.5-turbo").build();
        this.chatClient = chatClientBuilderOpenAI.defaultOptions(chatOptions).build();
        this.relevancyEvaluator = new RelevancyEvaluator(chatClientBuilderOpenAI);
        this.factCheckingEvaluator = new FactCheckingEvaluator(chatClientBuilderOpenAI);
    }

    @Override
    public String getResposta(String pergunta) {
        String resposta = chatClient.prompt().user(pergunta).call().content();

        EvaluationRequest evaluationRequest = new EvaluationRequest(pergunta, resposta);
        // EvaluationResponse response = relevancyEvaluator.evaluate(evaluationRequest);
        EvaluationResponse response = factCheckingEvaluator.evaluate(evaluationRequest);
        return resposta + "\n" +
            "Score: " + response.getScore() + "\n" +
            "Passou: " + response.isPass() + "\n";
    }
}
```